Experiment-4

AIM- To interface the HTS221 sensor via I2C and acquire temperature and humidity measurements.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX, Realterm.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. **Clock Selection**: Go to RCC and configure clock as "By pass Clock Source" as shown below:

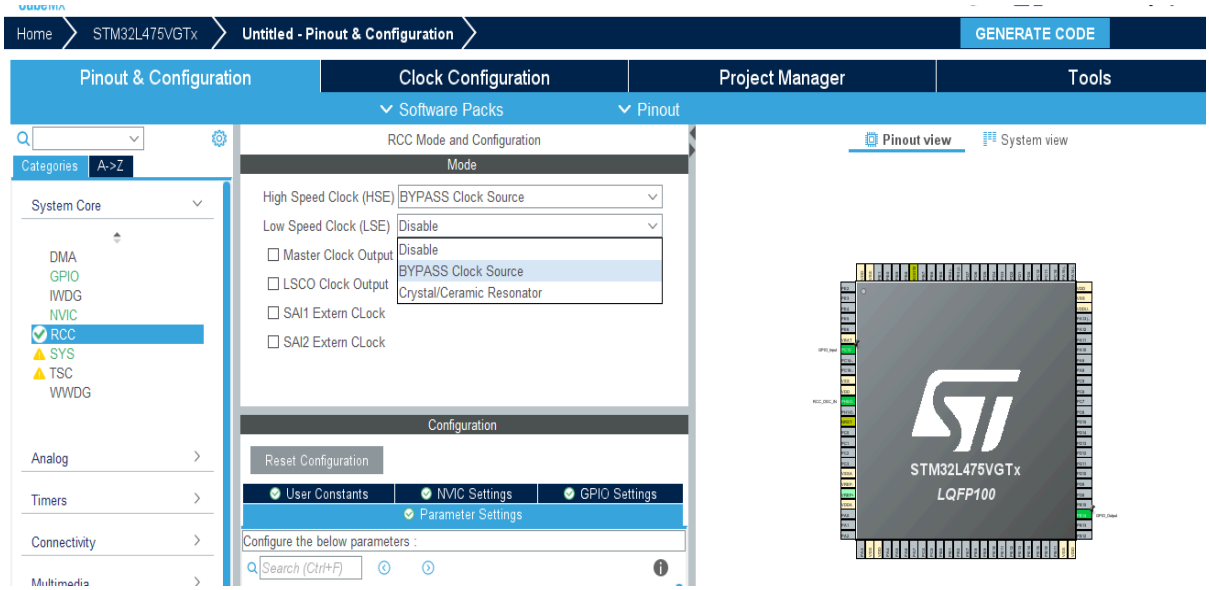

Fig-4.1 – Clock Selection

4. As per user manual of board ST LINK is internally connected to PB6(Tx) and PB7(Rx) of USART1.

Ayush
2310994514

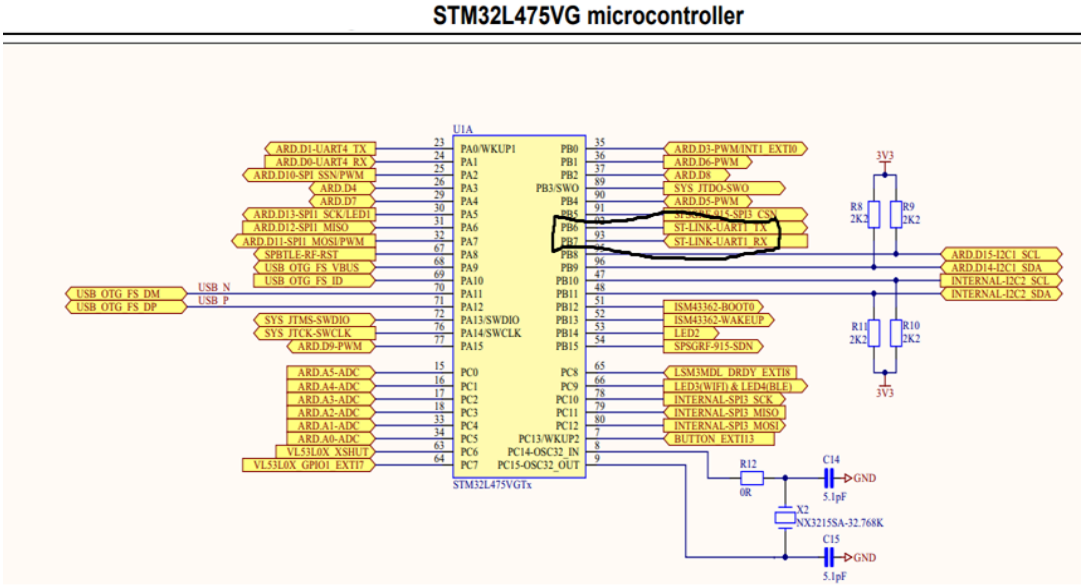

Fig-4.2 – STM32 Board Pin Configuration

5. Under “connectivity” click on “ USART1” and make PB6 as USART1_Tx and PB7 as USART2_Rx as shown below:

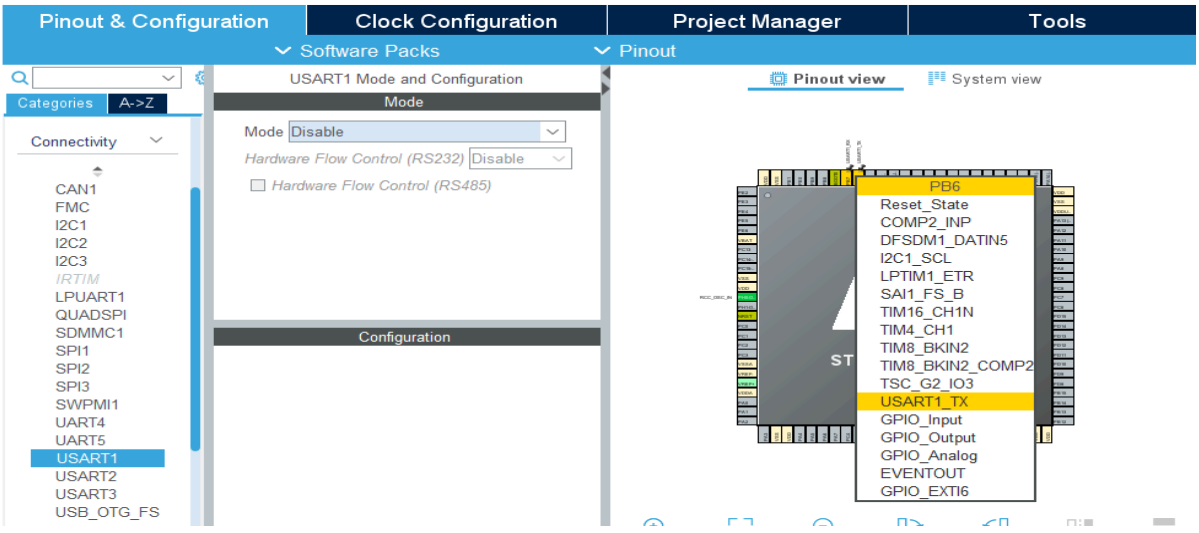

Fig-4.3 – STM32 Board Pinout and Configuration

6. Select USART1 and click on “ Asynchronous” in “Mode” as shown below :

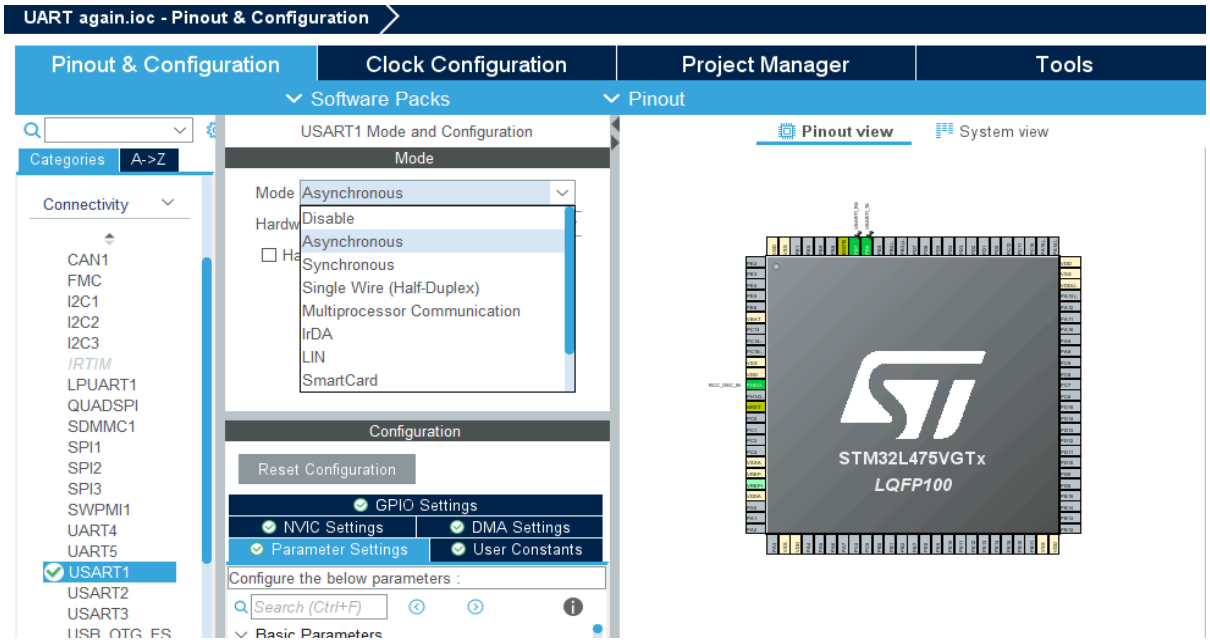

Fig-4.4 – Selection of Asynchronous mode

7. Go to "connectivity" and also select I2C1 in mode.
8. Click on PB8 and select I2C1_SCL to make PB8 pin as clock source pin as shown below:

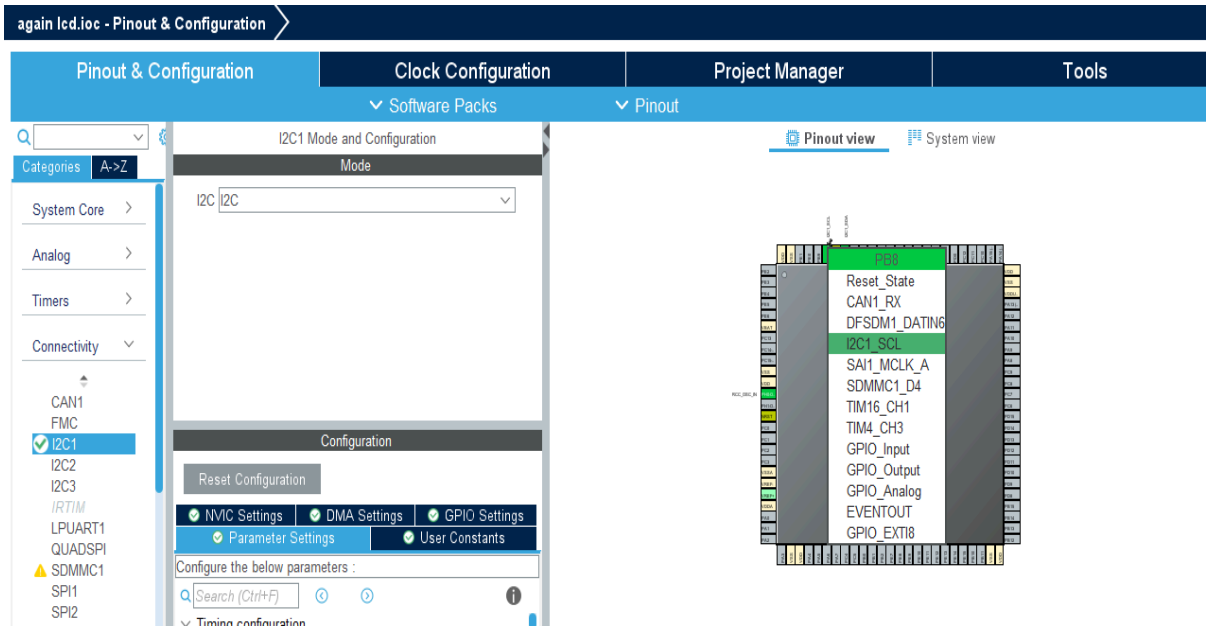

Fig-4.5 – Settings of Basic Parameters

9. **Clock Configuration:** Configure the clock as shown below to get a maximum frequency of 80MHz.

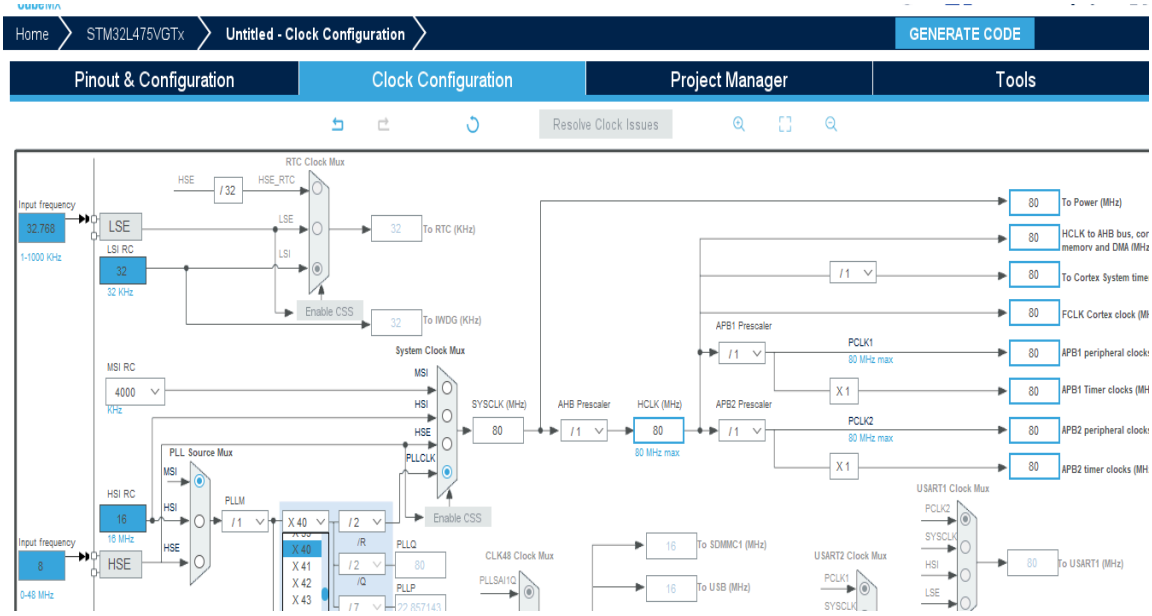

Fig-4.6 – Clock Configuration

10. Write Project name and select CubeIDE.
11. Click on generate code.
12. Click on open project.
13. Click ‘OK’ on the generated popup showing “successfully imported the project on the work space” and the project will be successfully added to the cube IDE.
14. Copy BSP drivers to your project.
15. The last version of the STM32CubeL4 package is downloaded by default in the STM32CubeMX repository (*C:\Users\user_name\STM32Cube\Repository\STM32Cube_FW_L4_Vx.xx.x\Drivers*).

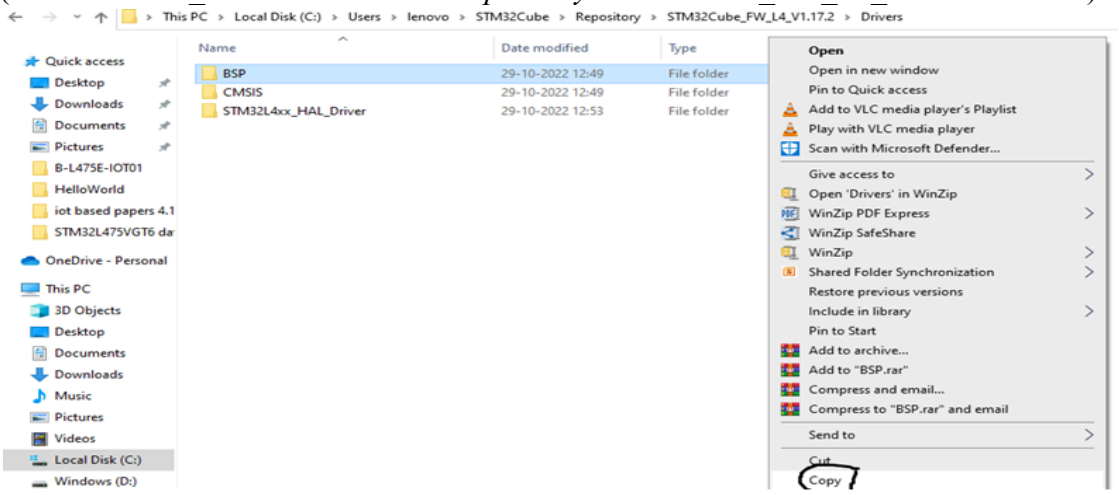

Fig-4.7 – BSP drivers location

16. Paste the copied BSP drivers in the drive folder as shown below:

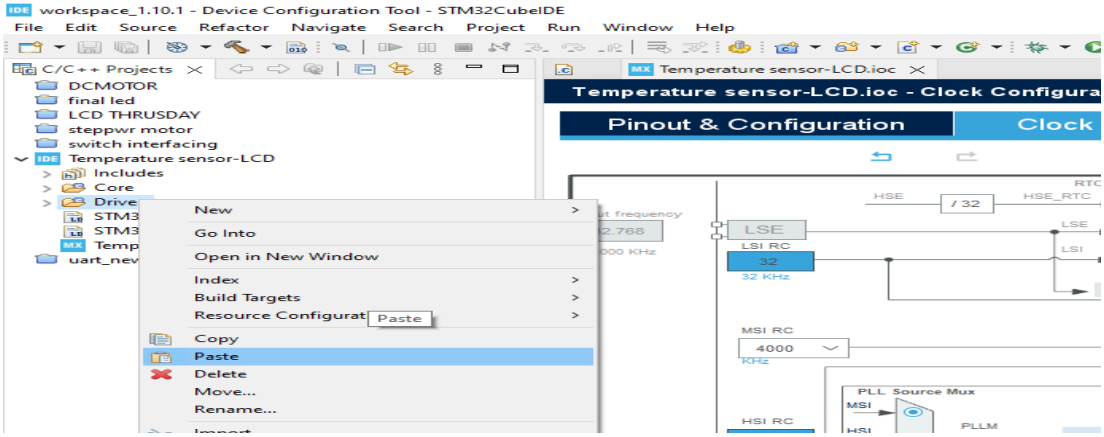

Fig-4.8 – Paste BSP drivers in project folder

17. Delete all folders in BSP, except B-L475E-IOT01 and the component as shown:

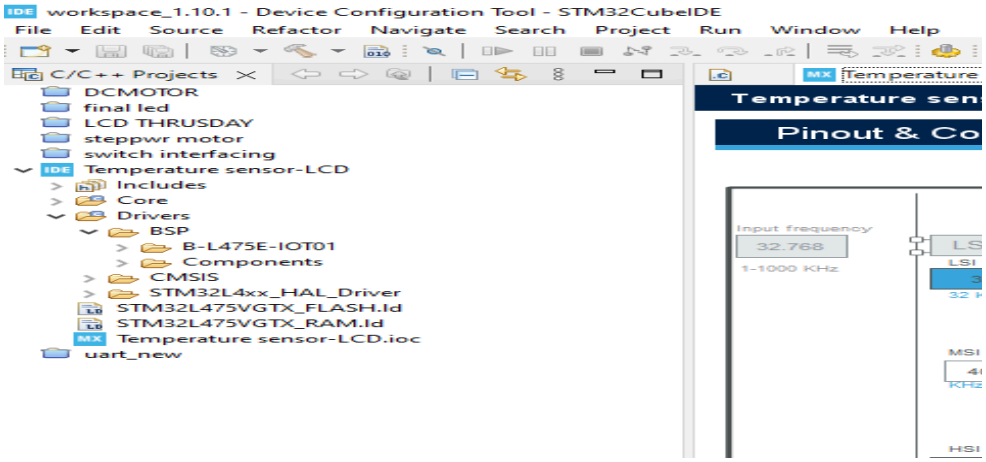

Fig-4.9 – B-L475E-IOT01 and component folder in BSP drivers

18. Update include paths: From Project menu or File menu, go to Properties > C/C++ Build > Settings > Tool Settings > MCU GCC Compiler > Include paths and click “add” icon.png to include the new paths as shown below:

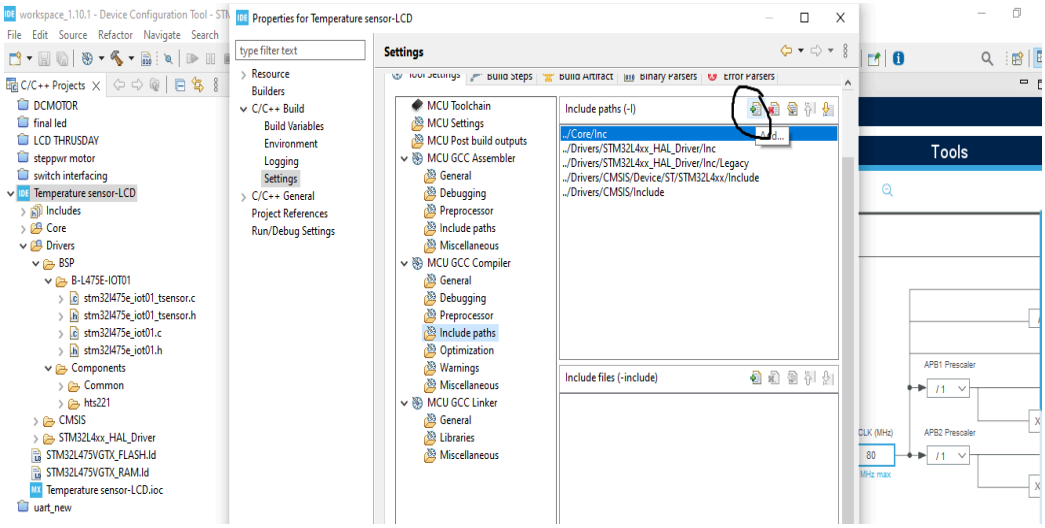

Fig-4.10 – Path settings

19. Add ../Drivers/BSP/B-L475E-IOT01 and ../Drivers/BSP/Components/hts221 paths as addresses given below:
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\B-L475E-IOT01“ and
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\Components\hts221

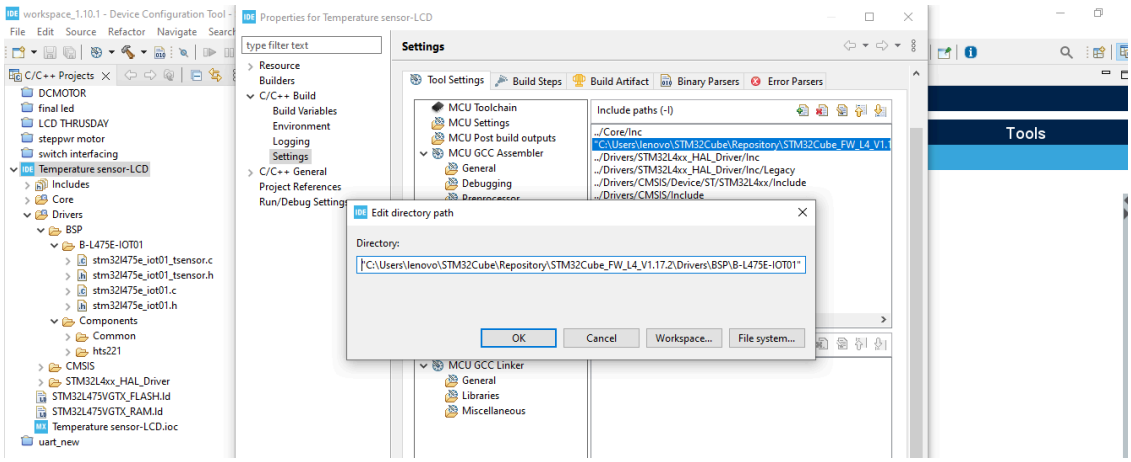

Fig-4.11 – Folders Path settings

20. After adding both path click “ apply and close” as shown below:

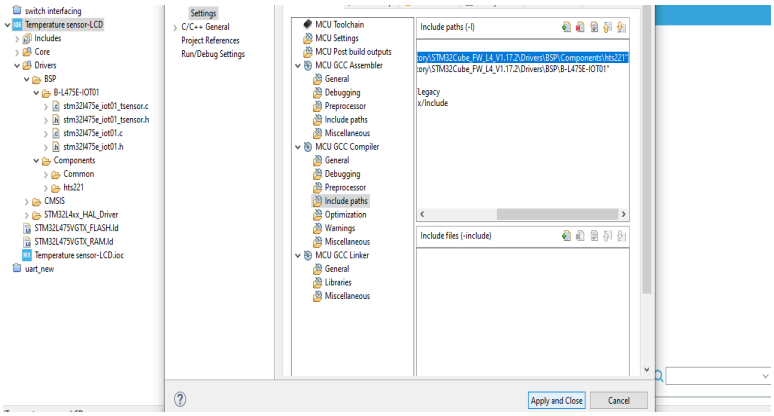

Fig-4.12 – Apply and close setting

21. Then "Build" and go to main.c file.

22. Include the header files in main.c files.

```
/* USER CODE BEGIN Includes */
#include "stm32l475e_iot01.h"
#include "stm32l475e_iot01_hsensor.h"
#include "stm32l475e_iot01_tsensor.h"
#include "stdlib.h"
#include <math.h>
#include <string.h>
#include <stdio.h>
```

23. Write the following instructions:

```
/* USER CODE BEGIN PV */
char str[50];
float temp_value = 0;
float hum_value = 0;// Measured temperature value
char str_tmp[100] = ""; // Formatted message to display the temperature value
uint8_t msg3[] = "=====> Temperature sensor HTS221 initialized \r\n ";
```

24. Write the following instructions as given:

```
/* USER CODE BEGIN 2 */
if (BSP_HSENSOR_Init() != HSENSOR_OK) {  // Use HSENSOR for both!
  uint8_t error[] = "SENSOR INIT FAILED!\r\n";
 HAL_UART_Transmit(&huart1, error, sizeof(error)-1, 1000);
 } else {
 uint8_t ok[] = "HTS221 OK - T+H Working\r\n";
 HAL_UART_Transmit(&huart1, ok, sizeof(ok)-1, 1000);
  }
  HAL_UART_Transmit(&huart1, msg3, sizeof(msg3), 1000);
  /* USER CODE BEGIN 2 */
  BSP_TSENSOR_Init();
  BSP_HSENSOR_Init();
HAL_UART_Transmit(&huart1,msg3,sizeof(msg3),1000);
 /* USER CODE END 2 */
```

25. Write the following instructions in while (1) loop

```
float hum_value = BSP_HSENSOR_ReadHumidity();
```

```
float temp_value = BSP_TSENSOR_ReadTemp();  // ← SAME SENSOR!
 // Debug both values
sprintf(str, "Temperature:%.1fC Humidity:%.0f%%\r\n", temp_value, hum_value);
HAL_UART_Transmit(&huart1, (uint8_t*)str, strlen(str), 1000);
HAL_Delay(1000);
```

26. Now connect the board and click on “Build”, “debug”.
27. Click on “Switch” on popup and click on “Resume”, the program will start execute.
28. Now, open the console “Real term” and set the baud rate as used in program (in this program it is 115200), select port (as per use) and click on “change” on the real term console.

OBSERVATION: The output value of temperature and humidity will display on real term as shown below:

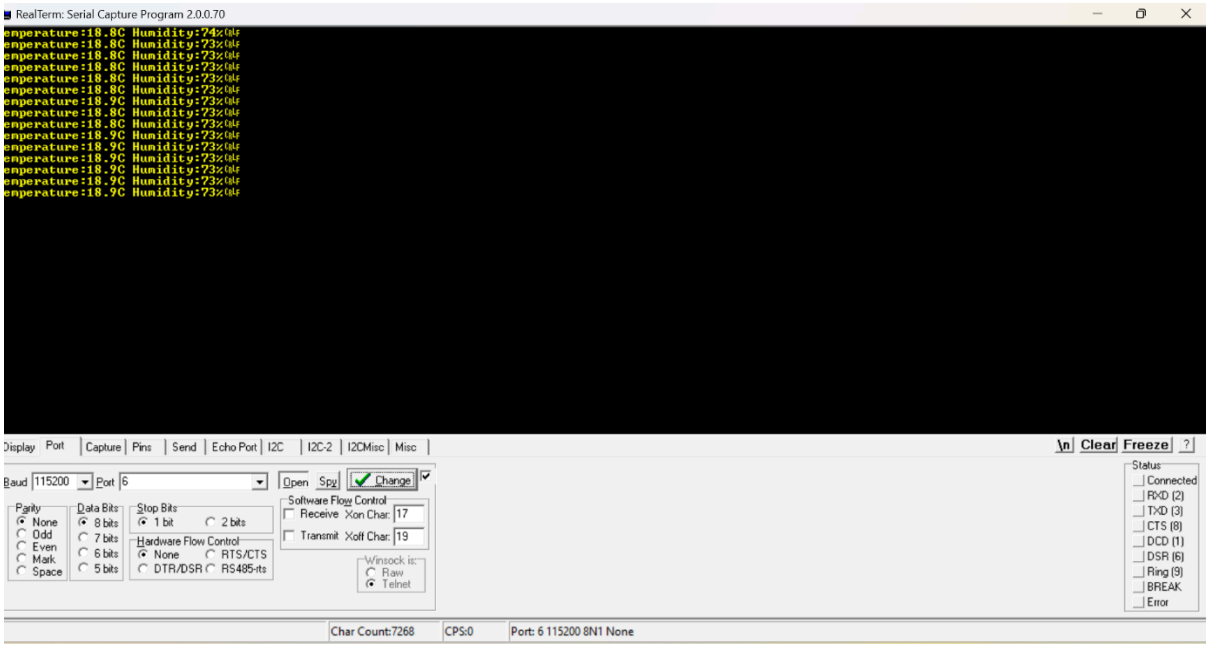

Fig: 4.13 – Output from experiment

Reflection Summary

Q1. Briefly explain how the HTS221 sensor is interfaced with the STM32L475 board using I2C and why I2C is suitable for this application (in terms of wiring and data exchange).

In this experiment, the HTS221 temperature and humidity sensor is interfaced with the STM32L475 board using the I2C protocol. The I2C1 peripheral of the STM32 is enabled in CubeMX and the SCL and SDA pins are configured. The STM32 works as the master and the HTS221 works as the slave device. Using the BSP functions, the microcontroller sends requests to the sensor over I2C and reads the temperature and humidity values from its registers.

I2C is suitable for this application because it uses only two wires (SDA and SCL), which reduces wiring complexity. It also supports address-based communication, so multiple devices can be connected on the same bus. Since temperature and humidity data do not change very fast, the speed of I2C is more than enough for this application.

Q2. In this experiment, what was the role of the BSP drivers and include-path configuration, and what kind of error or issue would you expect if these steps were not done correctly?

The BSP (Board Support Package) drivers provide ready-made functions to initialize and read data from the HTS221 sensor, such as reading temperature and humidity directly without writing low-level I2C code. This makes the program simpler and easier to understand.

The include-path configuration is required so that the compiler can find the header files of these BSP and sensor drivers during compilation.
If these steps are not done correctly, the project will show errors during compilation, such as "header file not found" or "undefined reference" to BSP functions. Even if the code compiles somehow, the sensor may not initialize or work properly at runtime.

Q3. Describe the complete data flow from sensing to display: how temperature and humidity values are acquired from HTS221, processed in the code, and finally visualized in Realterm using USART1

First, the HTS221 sensor continuously measures the surrounding temperature and humidity. This sensor is connected to the STM32L475 microcontroller through the I2C interface. When the program runs, the microcontroller communicates with the HTS221 using I2C and requests the latest sensor data.

In the code, the BSP driver functions BSP_TSENSOR_ReadTemp() and BSP_HSENSOR_ReadHumidity() are used. These functions internally perform the required I2C read operations from the sensor's registers and return the actual temperature (in °C) and humidity (in %) values to the microcontroller. The received values are stored in variables (temp_value and hum_value) inside the program.

After getting these values, the microcontroller processes and formats them into a readable text message using the sprintf() function. For example, the data is converted into a string like:
"Temperature: 25.3 C Humidity: 60 %".
This step is important because raw numerical values cannot be directly sent as a meaningful message to the serial terminal.

Next, this formatted string is transmitted from the STM32 to the PC using USART1 with the function HAL_UART_Transmit(). The USART1 is internally connected to

the ST-LINK virtual COM port, so whatever data the microcontroller sends over USART appears on the PC as serial data.

Finally, on the PC side, the Realterm software is opened and configured with the correct COM port and baud rate (115200). Realterm receives the serial data sent by the STM32 and displays the temperature and humidity values on the screen. This process repeats inside the while(1) loop with a delay, so the updated sensor readings are shown continuously in real time.

What the HTS221 is

The **HTS221** is a **capacitive digital sensor** used to measure **relative humidity and temperature**.

1. Package: HLGA-6L (2 × 2 × 0.9 mm), top-holed cap land grid array.
2. Operating temperature range: –40 °C to +120 °C
3. Low-power consumption: 2 µA @ 1 Hz ODR
4. High rH sensitivity: 0.004% rH/LSB
5. Accuracy:
    - Humidity: ±3.5% rH (20 to +80% rH)
    - Temperature: ±0.5 °C (15 to +40 °C)